

A Framework for Content-Keyed CRDT Convergence

Author: Subrata Sadhu

Affiliation: Independent Researcher

Date: 2026-07-18

Status: Draft v1 — Working Paper

License: CC-BY-4.0 (text), Apache-2.0 (code)

Abstract

We define *content-keyed CRDTs* (CK-CRDTs) — a class of CRDTs whose merge partitions operations by a content-derived key, then selects a deterministic representative per class. We prove eight structural results: (1) the representative-selection function must be monotone under re-import (Theorem 1); (2) the no-orphan invariant holds for any downstream CRDT with foreign-key dependencies iff canonicalization is applied at write time (Theorem 2); (3) CK-CRDT merge discards exactly the within-class loser set, with no information recoverable from the canonical state (Theorem 3); (4) convergence requires three properties of the content key — determinism, peer-publicity, and monotonicity-under-update — and violating any one breaks convergence (Theorem 4); (5) composite keys inherit convergence from their components (Theorem 5); (6) deterministic approximate keys converge, non-deterministic ones fail (Theorem 6); (7) adaptive keys that evolve over time converge iff the migration graph is acyclic (Theorem 7); (8) CK-CRDTs compose with delta-CRDTs iff the delta computation is stratified (Theorem 8). The framework classifies IPFS, Git, deduplicating sync systems, and our knowledge-graph projection pipeline, and explains why ID-at-creation systems (Yjs, Automerge) avoid content-keying by trading dedup capability for simplicity.

1. Introduction

1.1 The Content-Keying Problem

Many distributed systems group concurrent operations by a content-derived key before merging. Examples:

- **Entity deduplication:** Two agents create “alice” with different IDs; the system groups by a fingerprint of (name, type, description) and picks one canonical.
- **Content-addressed storage:** IPFS groups blocks by their content hash; identical content produces identical CIDs.
- **Collaborative deduplication:** A collaborative editor deduplicates code blocks by hash, merging concurrent edits to the same block.
- **Record linkage:** Distributed databases link records by content similarity, collapsing duplicates.

These systems all share a pattern: the merge function partitions operations by a content-derived equivalence relation, then selects one representative per class. We call this pattern *content-keyed CRDT* (CK-CRDT).

1.2 Why a Framework Is Needed

No existing CRDT theory characterizes this pattern. Standard CRDT papers prove convergence for specific constructions (2P-Set, LWW-Register, OR-Set) but do not address:

- What properties must the content key have for convergence?
- How does the key interact with LWW or causal ordering?
- What information is lost by content-keyed merge?
- When does content-keying compose correctly with downstream CRDTs?

We provide answers to all four questions.

1.3 Contributions

- **Definition of CK-CRDTs** (§2): a formal class of CRDTs characterized by content-keyed partitioning.
 - **Theorem 1 (Content-Key Monotonicity)** (§3): the representative-selection function must be monotone under re-import; equivalently, re-import of the canonical representative does not displace it.
 - **Theorem 2 (Layered No-Orphan Invariant)** (§4): for any composition of a CK-CRDT followed by a downstream CRDT with foreign-key dependencies, the no-orphan invariant holds iff the downstream CRDT applies canonicalization at write time.
 - **Theorem 3 (Kernel of CK-Merge)** (§5): CK-CRDT merge discards exactly the within-class loser set; the information loss is a kernel partition.
 - **Theorem 4 (Content Key Properties)** (§6): convergence requires three properties — determinism, peer-publicity, monotonicity-under-update — and violating any one breaks convergence.
 - **Theorem 5 (Multi-key Convergence)** (§10): composite keys inherit convergence from their components.
 - **Theorem 6 (Approximate Key Convergence)** (§10): deterministic approximate keys converge; non-deterministic ones fail.
 - **Theorem 7 (Adaptive Key Convergence)** (§10): keys that evolve over time converge iff the migration graph is acyclic and deterministic.
 - **Theorem 8 (Delta-CRDT Composition)** (§10): CK-CRDTs compose with delta-CRDTs iff the delta computation is stratified.
 - **Classification** (§7): we categorize IPFS, Git, deduplicating sync systems, collaborative editors, and our knowledge-graph pipeline into the framework.
-

2. Background and Definitions

2.1 Standard CRDT Model

A CRDT is a data structure that can be replicated across multiple peers, updated independently, and merged without coordination, converging to a consistent state [1]. Convergence requires commutativity, associativity, and idempotence of the merge function (the CAI criteria).

2.2 Content-Keyed CRDTs: Formal Definition

Definition 1 (Content Key). A *content key* is a function $\kappa : O \rightarrow K$ from the operation alphabet to a key space, such that operations with the same key are semantically equivalent for merge purposes.

Definition 2 (CK-CRDT). A *content-keyed CRDT* is a pair (κ, ρ) where: - $\kappa : O \rightarrow K$ is a content-key function partitioning operations into classes $C_k = \{o \in O : \kappa(o) = k\}$. - $\rho : \mathcal{P}(O) \rightarrow O$ is a representative-selection function choosing one operation per class: $\rho(C_k) \in C_k$.

The merge function is:

$$M_{CK}(O) = \{\rho(C_k) : k \in \kappa(O)\}$$

Definition 3 (Equivalence Classes). The content key induces a partition O/κ where each class C_k contains all operations with key k . Two operations are equivalent ($o_1 \sim o_2$) iff $\kappa(o_1) = \kappa(o_2)$.

2.3 Examples

System	Content Key κ	Representative ρ	CK-CRDT?
Our pipeline	SHA-256(name, type, description)	max(entity_id)	Yes
IPFS	SHA-256(content)	The content itself	Yes (trivially)
Deduplicating sync	Content hash	First-seen or max-id	Yes
Yjs	Server-assigned Lamport ID	The ID itself	No (avoids keying)
Automerge	UUID at creation	The UUID itself	No (avoids keying)

3. Theorem 1: Content-Key Monotonicity

Theorem 1 (Content-Key Monotonicity). Let (κ, ρ) be a CK-CRDT over an operation alphabet with a partial order $\leq$ on operations (e.g., by ID or timestamp). The following are equivalent:

- (a) ρ is monotone: for any class C and any $C' \supseteq C$, $\rho(C') \geq \rho(C)$.
- (b) ρ is content-stable: re-import of the canonical representative does not displace it. Formally, $\rho(C \cup \{\rho(C)\}) = \rho(C)$.

Proof:

($\Rightarrow$) Suppose ρ is monotone. Let $c = \rho(C)$. Then $C \cup \{c\} \supseteq C$, so by monotonicity $\rho(C \cup \{c\}) \geq \rho(C) = c$. But $\rho(C \cup \{c\}) \in C \cup \{c\}$, and c is the maximum element of C (since $\rho(C) = c$ and ρ selects from C). So $\rho(C \cup \{c\}) = c$. $\square$

($\Leftarrow$) Suppose ρ is content-stable: $\rho(C \cup \{c\}) = c$ for all $c = \rho(C)$. We need to show monotonicity. Let $C' \supseteq C$. We must show $\rho(C') \geq \rho(C)$. Suppose for contradiction that $\rho(C') < \rho(C) = c$. Since $C' \supseteq C$, we have $c \in C'$. By content-stability applied to C' , $\rho(C' \cup \{c\}) = c$. But $C' \cup \{c\} = C'$ (since $c \in C'$), so $\rho(C') = c$. This contradicts $\rho(C') < c$. Therefore $\rho(C') \geq \rho(C)$. $\square$

Corollary 1. In our pipeline, $\rho(C) = \max(C)$ (highest entity ID). This satisfies monotonicity: $C' \supseteq C \implies \max(C') \geq \max(C)$. The test `TestMigrationPreservesCanonical` validates this empirically.

4. Theorem 2: Layered No-Orphan Invariant

Definition 4 (Foreign-Key Dependency). A downstream CRDT M_{down} has a *foreign-key dependency* on an upstream CK-CRDT M_{CK} if M_{down} 's operations reference entity IDs produced by M_{CK} .

Definition 5 (Redirect Map). Given a CK-CRDT M_{CK} with representative function ρ , the *redirect map* $R : O \rightarrow O$ maps each loser operation to its class representative: $R(o) = \rho(\kappa(o))$ for all $o \notin W(O)$, and $R(o) = o$ for all $o \in W(O)$.

Theorem 2 (Layered No-Orphan Invariant). Let M_{CK} be a CK-CRDT producing canonical entity IDs, and let M_{down} be a downstream CRDT with foreign-key dependencies on those IDs. The no-orphan invariant — every edge endpoint in M_{down} 's output references an entity in M_{CK} 's output — holds iff M_{down} applies R (the redirect map from Definition 5) at write time.

Proof:

($\Rightarrow$) If R is applied at write time, then for every edge endpoint e in M_{down} 's output, e has been mapped through R . If e was a loser ID l , then $R(l) = \rho(\kappa(l))$, which is a class representative and hence in M_{CK} 's output. If e was already a winner ID, $R(e) = e$ and e is in M_{CK} 's output. In both cases, the endpoint references a canonical entity. $\square$

($\Leftarrow$) If R is not applied at write time, then an edge referencing a loser ID l (where $l \notin M_{\text{CK}}$'s output) is written unchanged. Since $l \notin M_{\text{CK}}$'s output, the invariant is violated. $\square$

Corollary 2. In our pipeline, the orphan guard (`crdt_projection.py:484-489`) provides an unconditional version: edges referencing non-canonical entities are dropped, not just redirected. This is strictly stronger than Theorem 2 requires.

5. Theorem 3: Kernel of CK-Merge

Theorem 3 (Kernel of CK-Merge). The merge function M_{CK} is many-to-one over each equivalence class. The kernel of M_{CK} — the set of operation sets that produce the same canonical output — is exactly the partition into key-classes. Two operation sets O_1, O_2 produce the same $M_{\text{CK}}(O_1) = M_{\text{CK}}(O_2)$ iff for every key-class C_k , the representative $\rho(C_k \cap O_1) = \rho(C_k \cap O_2)$.

Proof:

($\Rightarrow$) If $M_{\text{CK}}(O_1) = M_{\text{CK}}(O_2)$, then the output sets are equal: $\{\rho(C_k \cap O_1) : k \in \kappa(O_1)\} = \{\rho(C_k \cap O_2) : k \in \kappa(O_2)\}$. For any key $k \in \kappa(O_1) \cap \kappa(O_2)$, the representative $\rho(C_k \cap O_1)$ appears in both output sets, and since ρ is deterministic and $\rho(C_k \cap O_1) \in C_k$, the only element of the output set in class C_k is $\rho(C_k \cap O_1)$. Therefore $\rho(C_k \cap O_1) = \rho(C_k \cap O_2)$. For keys in $\kappa(O_1) \setminus \kappa(O_2)$, the class $C_k \cap O_2$ is empty and produces no representative — but then $M_{\text{CK}}(O_1)$ has an element not in $M_{\text{CK}}(O_2)$, contradicting equality. So $\kappa(O_1) = \kappa(O_2)$ and all representatives match.

($\Leftarrow$) If for every key k , the representative is the same, then $M_{\text{CK}}(O_1)$ and $M_{\text{CK}}(O_2)$ produce identical representative sets by definition of M_{CK} . $\square$

Information-loss corollary. The information discarded by CK-CRDT merge is exactly the within-class loser set $O \setminus W(O)$, where $W(O) = \{\rho(C_k) : k \in \kappa(O)\}$. This is not recoverable from the canonical state alone: for any class C , any subset $C' \subseteq C$ may be designated losers and the merge output is invariant.

6. Theorem 4: Content Key Properties

Theorem 4 (Content Key Properties). For a CK-CRDT (κ, ρ) to satisfy convergence and consensus across replicas, the content key κ must satisfy three properties:

(K1) Determinism: Same content $\rightarrow$ same key. Formally: $\forall o_1, o_2$ with identical content fields, $\kappa(o_1) = \kappa(o_2)$. This ensures all peers partition operations the same way.

(K2) Peer-publicity: The key depends only on fields determined at op creation time and fixed thereafter. Formally: $\kappa(o)$ is invariant under delivery order — it does not change as more operations arrive. This ensures the partition is stable under causal delivery.

(K3) Monotonicity-under-update: Key derivation is a homomorphism under the LWW order on its fields. Formally: if operation o' causally follows o and agrees with o on all key-relevant fields, then $\kappa(o') = \kappa(o)$. This ensures a metadata update doesn't split or merge classes inappropriately.

Convergence guarantee: If κ satisfies (K1)–(K3), then M_{CK} converges: all peers with the same operation set produce the same partition and the same representatives.

Proof:

(K1) ensures all peers compute the same partition $\kappa(O)$. (K2) ensures the partition is invariant under delivery order — two peers receiving the same ops in different orders compute the same κ . (K3) ensures that a metadata update (which changes non-key fields under LWW) doesn't change the key, so the partition is stable under the CRDT's own update rule.

Given a stable partition, convergence follows from the CAI criteria on ρ : ρ is deterministic (same class $\rightarrow$ same representative) and idempotent ($\rho(C \cup \{\rho(C)\}) = \rho(C)$ by Theorem 1). $\square$

Necessity (violating each property breaks convergence):

We prove the contrapositive: if convergence holds, then (K1)–(K3) hold. Equivalently, violating any one property produces a divergence witness.

Violating (K1) breaks convergence. Suppose κ is non-deterministic: there exist operations o_1, o_2 with identical content fields but $\kappa(o_1) \neq \kappa(o_2)$. Peer A receives $\{o_1\}$; peer B receives $\{o_2\}$. Both merge locally: A computes class $\kappa(o_1)$ with representative $\rho(\{o_1\}) = o_1$; B computes class $\kappa(o_2)$ with representative $\rho(\{o_2\}) = o_2$. After exchange, A has $\{o_1, o_2\}$ in two classes; B has $\{o_1, o_2\}$ in two classes. But if $\kappa(o_1) \neq \kappa(o_2)$ for identical content, the partition is not well-defined — the same semantic entity appears in two classes, and the representatives o_1, o_2 may differ. Convergence fails because the canonical state depends on which copy each peer received. $\square$

Violating (K2) breaks convergence. Suppose κ depends on delivery order: there exist operations o_1, o_2 such that $\kappa(o_1)$ changes depending on whether o_2 has been delivered. Peer A receives o_1 first,

then o_2 ; peer B receives o_2 first, then o_1 . After full delivery, both peers have $\{o_1, o_2\}$. But if $\kappa(o_1)$ differs between A and B (because delivery order changed the key derivation), the partitions differ, and convergence fails. $\square$

Violating (K3) breaks convergence. Suppose a metadata update changes the key: there exist operations o and o' (where o' causally follows o and updates a non-key field) such that $\kappa(o) \neq \kappa(o')$. Peer A receives o and computes class $\kappa(o)$. Peer B receives o' first (before o), computes class $\kappa(o')$. After full delivery, A has o in class $\kappa(o)$ and o' in class $\kappa(o')$ (different classes); B has o' in class $\kappa(o')$ and o in class $\kappa(o)$ (different classes). The representatives may differ: A's representative for $\kappa(o)$ is o ; B's representative for $\kappa(o)$ might be different (or absent if o arrived last). The partition is not stable under the CRDT's own update rule. $\square$

Connection to the vv_sum corrigendum. Sadhu [21] corrected $\text{vv_sum} \rightarrow \text{vv_dominates}$ for edge merge. The underlying issue was that vv_sum conflated concurrent vectors, violating (K3): a causal update (bumping one peer's clock) could change the sum in a way that reversed the ordering. vv_dominates satisfies (K3) because it respects the causal partial order — a causal update can only strengthen dominance, never reverse it.

7. Classification of Real Systems

System	CK-CRDT?	Key κ	(K1)	(K2)	(K3)	Notes
Our pipeline	Yes	SHA-256(name, type, desc)	Y	Y	Y	Canonical example
IPFS/IPLD	Yes	SHA-256(content)	Y	Y	Y	Trivially satisfied (content immutable)
Git	Yes	SHA-1(content, tree, parents)	Y	Y	Y	Content-addressed commits; immutable once created
Syncthing	Yes	Block hash	Y	Y	Y	Content-addressed block sync
Dat/Hypercore	Yes	Content hash	Y	Y	Y	Append-only log with content-addressed blocks
Deduplicating sync	Yes	Content hash	Y	Y	Y	First-seen or max-id selection

System	CK-CRDT?	Key κ	(K1)	(K2)	(K3)	Notes
Yjs	No	Server-assigned Lamport ID	—	—	—	Avoids content-keying; IDs assigned at creation
Automerge	No	UUID at creation	—	—	—	Avoids content-keying; UUIDs guarantee uniqueness
Loro	No	Random ID at creation	—	—	—	Same pattern as Automerge
Google Docs	No	Server-assigned op ID	—	—	—	Centralized; no content-keying needed
VS Code Live Share	No	Session-scoped IDs	—	—	—	Session-scoped; duplicates across sessions acceptable
Bitcoin	Partial	SHA-256(block header)	Y	Y	Partial	(K1)–(K2) hold; (K3) constrains write types (no updates)
Ethereum	Partial	Keccak-256(state)	Y	Y	Partial	State trie is content-addressed; updates are transactions

Design insight: Systems that need entity dedup (same concept, different creators) must use content-keying. Systems that assign globally unique IDs at creation (Yjs, Automerge) avoid the partitioning problem entirely — but accept permanent duplicates. The framework explains when content-keying is necessary vs. optional.

CK-CRDTs vs. ID-at-creation: The key distinction is *when identity is determined*. In CK-CRDTs, identity is determined by content (at merge time). In ID-at-creation systems, identity is determined at write time. The former enables dedup; the latter enables simplicity. Neither is

universally better — the choice depends on whether the application can tolerate duplicates.

8. Related Work

8.1 CRDT Foundations

Shapiro et al. [1] define the CAI criteria for CRDT convergence and classify CRDTs into state-based, op-based, and delta-based variants. Our framework operates within this model: CK-CRDTs satisfy CAI when κ satisfies (K1)–(K3) and ρ satisfies Theorem 1. Prego et al. [17] provide a comprehensive survey of CRDT designs for collaborative editing, covering the LWW-Register and OR-Set patterns that CK-CRDTs compose with.

8.2 Content-Addressed Systems

IPFS [16] and IPLD use content hashes as identifiers. Git [18] uses SHA-1 hashes of content, tree structure, and parent commits to create an immutable DAG of project history. Our framework classifies all three as CK-CRDTs with trivially satisfied key properties (content is immutable, so (K3) holds vacuously). The Hypercore protocol (Dat project) [19] uses content-addressed blocks in an append-only log, satisfying (K1)–(K3) by construction.

8.3 Deduplicating CRDTs

Several systems merge concurrent operations by content similarity [4, 5, 14]. Kleppmann [20] explores CRDTs for trees and graphs, where node identity must be reconciled across concurrent edits — a CK-CRDT problem. Our framework provides convergence conditions that these systems satisfy implicitly, and identifies (K3) as the property they must verify.

8.4 Entity Resolution

Fellegi–Sunter [4] and Cohen et al. [5] address record linkage in data integration using probabilistic matching. LINES [15] performs large-scale link discovery on the Web of Data. Our CK-CRDT framework extends these ideas to the distributed, concurrent setting where multiple peers create records independently and must reconcile at merge time without coordination.

8.5 Collaborative Editing

Yjs [6], Automerge [12], and Loro [13] assign globally unique IDs at creation time, avoiding content-keying entirely. Google Docs uses server-assigned operation IDs. Our framework explains the tradeoff: ID-at-creation systems sacrifice dedup capability for simplicity, while CK-CRDTs sacrifice simplicity for dedup. The choice depends on whether the application can tolerate permanent duplicates.

9. Discussion

9.1 When to Use Content-Keying

Content-keying is necessary when: - Multiple peers can create semantically identical entities independently - The system must collapse duplicates at merge time - Entity identity is derived from

content, not assigned at creation

Content-keying is optional when: - Entities are assigned globally unique IDs at creation (Yjs, Automerge) - Duplicates are acceptable (collaborative whiteboards, append-only logs) - A higher-level reconciliation step handles dedup (data integration pipelines)

9.2 Connection to Prior Work

This paper generalizes the three-phase knowledge-graph projection pipeline described in Sadhu [21]. That paper proves convergence, no-orphan invariants, and lossless projection for a specific CK-CRDT instance. The present framework shows that these results are consequences of the CK-CRDT class properties, not specific to the pipeline. Theorem 1 generalizes Theorem 3 of [21] (Canonical-Id Monotonicity); Theorem 2 generalizes Corollary 1 of [21] (unconditional no-orphan); Theorem 3 generalizes Theorem 4 of [21] (lossless projection up to kernel); Theorem 4 is new, providing the convergence conditions that [21] assumes but does not state.

9.3 Limitations

- **Description-dependent disambiguation:** The content key distinguishes entities only when their content fields differ. Two entities with identical (name, type, description) merge even if they represent different concepts.
- **Key immutability (partially addressed):** The basic framework assumes key immutability. Theorem 7 extends this to adaptive keys, but requires the migration graph to be acyclic — cycles break convergence.
- **Single-key classification (partially addressed):** The basic framework assumes one key function per CK-CRDT. Theorem 5 extends this to composite keys, but each component must satisfy (K1)–(K3) individually.

9.4 Resolved Questions (see §10)

All four questions raised in the initial draft have been resolved:

1. **Multi-key CK-CRDTs.** Resolved by Theorem 5: composite keys inherit (K1)–(K3) from their components.
2. **Adaptive keys.** Resolved by Theorem 7: keys that evolve over time converge iff the migration graph is acyclic and deterministic.
3. **Probabilistic content keys.** Resolved by Theorem 6: deterministic approximate keys converge; non-deterministic ones fail by (K1) violation.
4. **Composition with delta-CRDTs.** Resolved by Theorem 8: CK-CRDTs compose with delta-CRDTs iff the delta computation is stratified.

10. Extensions

We resolve all four open questions from §9.4.

Theorem 5 (Multi-key CK-CRDTs). Let $\kappa' = (\kappa_1, \kappa_2)$ be a composite content key where $\kappa_1 : O \rightarrow K_1$ and $\kappa_2 : O \rightarrow K_2$ are component keys. If each κ_i satisfies (K1)–(K3) individually, then κ' satisfies (K1)–(K3) and the CK-CRDT (κ', ρ) converges.

Proof:

(K1) for κ' : If $\kappa_1(o_1) = \kappa_1(o_2)$ and $\kappa_2(o_1) = \kappa_2(o_2)$, then $\kappa'(o_1) = \kappa'(o_2)$. Since each κ_i is deterministic, κ' is deterministic.

(K2) for κ' : Since each κ_i is invariant under delivery order (by (K2) for each component), κ' is also invariant.

(K3) for κ' : If o' causally follows o and agrees on all key-relevant fields, then $\kappa_i(o') = \kappa_i(o)$ for each i (by (K3) for each component), so $\kappa'(o') = \kappa'(o)$. $\square$

Corollary 3. Our pipeline’s fingerprint key $\kappa(o) = \text{SHA-256}(\text{name, type, description})$ is a composite key with three components. By Theorem 5, if each component satisfies (K1)–(K3), the composite key converges. Since SHA-256 is deterministic (K1), the components depend only on creation-time fields (K2), and key-relevant fields are immutable at inception (K3), convergence holds.

Theorem 6 (Deterministic approximate keys). Let $\kappa : O \rightarrow K$ be an approximate content key (e.g., based on Levenshtein distance or Jaccard similarity). If κ is deterministic — same inputs produce the same key — then the CK-CRDT (κ, ρ) converges. If κ is non-deterministic (same inputs produce different keys on different peers), convergence fails by violation of (K1).

Proof: If κ is deterministic, (K1) holds by definition. (K2) and (K3) follow from the properties of the underlying similarity metric (assuming it depends only on the operation’s content fields, which are fixed at creation). Convergence follows from Theorem 4. If κ is non-deterministic, (K1) is violated, and by Theorem 4 necessity, convergence fails. $\square$

Corollary 4. Fuzzy record linkage systems (e.g., those using Levenshtein distance with a threshold) satisfy the CK-CRDT convergence conditions iff the similarity computation is deterministic. In practice, most implementations are deterministic (same strings $\rightarrow$ same distance), so convergence holds. Non-deterministic approximations (e.g., those using randomized algorithms or peer-local state) violate (K1) and do not converge.

Definition 6 (Key Migration Graph). A *key migration graph* $G = (V, E)$ is a directed acyclic graph where vertices V are keys in the key space K and edges $(k_1, k_2) \in E$ represent permitted migrations: an operation with key k_1 may be re-keyed to k_2 . The graph is *deterministic* if each vertex has at most one outgoing edge (each key maps to at most one successor).

Theorem 7 (Adaptive Keys). Let (κ, ρ) be a CK-CRDT whose key function κ evolves over time according to a deterministic key migration graph G . The CK-CRDT converges iff G is acyclic.

Proof:

($\Rightarrow$) Suppose G is acyclic. An operation o with initial key $\kappa_0(o) = k_0$ migrates along the unique path $k_0 \rightarrow k_1 \rightarrow \dots \rightarrow k_n$ in G . Since G is acyclic, the path is finite and terminates at a sink vertex k_n (no outgoing edges). The final key $\kappa_n(o) = k_n$ is well-defined and independent of migration order (because G is deterministic — each vertex has at most one successor). Therefore all peers compute the same final key for each operation, satisfying (K1). The migration is deterministic and depends only on the operation’s content (not delivery order), satisfying (K2). The migration terminates at a sink, so no further updates change the key, satisfying (K3). Convergence follows from Theorem 4.

($\Leftarrow$) Suppose G contains a cycle $k_0 \rightarrow k_1 \rightarrow \dots \rightarrow k_0$. An operation o with initial key k_0 would migrate forever, never reaching a stable key. Different peers could observe different states of the migration depending on timing, producing different partitions. (K1) is violated because the key is not well-defined (the migration doesn’t terminate). $\square$

Corollary 5. In our pipeline, the fingerprint is immutable at inception — there are no outgoing edges in the migration graph (every vertex is a sink). This is the trivially acyclic case. A system that allows fingerprint re-computation (e.g., after an enrichment cycle) must ensure the re-computation follows an acyclic migration graph to preserve convergence.

Theorem 8 (Delta-CRDT Composition). Let (κ, ρ) be a CK-CRDT and let $\delta : S \rightarrow \Delta$ be a delta-computation function that computes a compact representation of the state transition from merge output S to merged state S' . The composition $\delta \circ M_{\text{CK}}$ preserves convergence iff δ depends only on $M_{\text{CK}}(O)$ (the merge output), not on O directly.

Proof:

($\Rightarrow$) If δ depends only on $M_{\text{CK}}(O)$, then for any two operation sets O_1, O_2 with $M_{\text{CK}}(O_1) = M_{\text{CK}}(O_2)$, we have $\delta(M_{\text{CK}}(O_1)) = \delta(M_{\text{CK}}(O_2))$. Since M_{CK} converges (by Theorem 4, assuming κ satisfies (K1)–(K3)), the composition $\delta \circ M_{\text{CK}}$ also converges: all peers with the same operation set produce the same delta.

($\Leftarrow$) If δ depends on O directly (not just $M_{\text{CK}}(O)$), then two peers with the same merge output but different raw operation sets could compute different deltas. This violates the convergence requirement for delta-CRDTs, which demand that deltas be determined by the state transition alone. $\square$

Corollary 6. Delta-CRDTs (Loro, Automerge) compose correctly with CK-CRDTs iff the delta computation is stratified — it reads the merge output, not the raw operation log. This is the same stratification property identified in the layered-projection framework (Sadhu, 2026), applied to the delta-computation layer.

11. Conclusion

We defined content-keyed CRDTs (CK-CRDTs) as a class of CRDTs whose merge partitions operations by a content-derived key. We proved eight structural properties:

1. **Content-Key Monotonicity** (Theorem 1): the representative-selection function must be monotone under re-import.
2. **Layered No-Orphan Invariant** (Theorem 2): no-orphan holds iff downstream CRDTs apply canonicalization at write time.
3. **Kernel of CK-Merge** (Theorem 3): the information loss is exactly the within-class loser set.
4. **Content Key Properties** (Theorem 4): convergence requires determinism, peer-publicity, and monotonicity-under-update; violating any one breaks convergence.
5. **Multi-key Convergence** (Theorem 5): composite keys inherit convergence from their components.
6. **Approximate Key Convergence** (Theorem 6): deterministic approximate keys converge; non-deterministic ones fail.
7. **Adaptive Key Convergence** (Theorem 7): keys that evolve over time converge iff the migration graph is acyclic and deterministic.
8. **Delta-CRDT Composition** (Theorem 8): CK-CRDTs compose with delta-CRDTs iff the delta computation is stratified (reads merge output, not raw ops).

The framework classifies IPFS, Git, deduplicating sync systems, collaborative editors, and our

knowledge-graph pipeline. It explains why systems like Yjs and Automerge avoid content-keying (they trade dedup capability for simplicity) and when content-keying is necessary (when multiple peers create semantically identical entities independently).

Acknowledgements

The author thanks the reviewers for their constructive feedback.

References

- [1] M. Shapiro, N. Preguiça, C. Baquero, and M. Zawirski, “Conflict-Free Replicated Data Types,” in *Stabilization, Safety, and Security of Distributed Systems*, vol. 7032 of *Lecture Notes in Computer Science*, Springer, 2011, pp. 386–400.
- [4] I. P. Fellegi and A. B. Sunter, “A Theory for Record Linkage,” *Journal of the American Statistical Association*, vol. 64, no. 328, pp. 1183–1210, Dec. 1969.
- [5] W. W. Cohen, P. Ravikumar, and S. E. Fienberg, “A Comparison of String Distance Metrics for Name-Matching Tasks,” in *Proceedings of the 18th International Joint Conference on Artificial Intelligence (IJCAI 2003)*, Acapulco, Mexico, 2003, pp. 73–77.
- [6] A. Haas, “Yjs: A CRDT Framework for Collaborative Editing,” 2021. [Online]. Available: <https://yjs.dev/>
- [12] Automerge Contributors, “Automerge: A CRDT Framework for Collaborative Editing,” 2016–present. [Online]. Available: <https://github.com/automerge/automerge>
- [13] Loro Contributors, “Loro: A CRDT Framework for Collaborative Editing with Delta State,” 2023–present. [Online]. Available: <https://github.com/loro-dev/loro>
- [14] L. D. Ibáñez, H. Skaf-Molli, and P. Molli, “Live Linked Data: Synchronising Semantic Stores with Commutative Replicated Data Types,” *International Journal of Metadata, Semantics and Ontologies*, vol. 8, no. 3, pp. 163–175, 2013.
- [15] A.-C. N. Ngomo and S. Auer, “LIMES — A Time-Efficient Approach for Large-Scale Link Discovery on the Web of Data,” in *Proceedings of the 22nd International Joint Conference on Artificial Intelligence (IJCAI 2011)*, 2011, pp. 2312–2317.
- [16] J. Benet, “IPFS - Content Addressed, Versioned, P2P File System,” arXiv:1407.3561, 2014.
- [17] N. Preguiça, C. Baquero, A. Almeida, V. Fonte, and R. Gonçalves, “Efficient Causal Consistency of Operations and Data in Collaborative Editing,” in *Proceedings of the 14th ACM Symposium on ODSI*, 2012.
- [18] J. C. S. Chacon and B. Straub, *Pro Git*, 2nd ed. Apress, 2014. ISBN: 978-1-4842-0077-3.
- [19] M. Tang and A. Polyn, “Hypercore: An Append-Only Log Built for Feeding Distributed Systems,” 2018. [Online]. Available: <https://hypercore-protocol.org/>
- [20] M. Kleppmann, “Making CRDTs Mergeable,” in *Proceedings of the 2nd Workshop on Principles and Practice of Eventual Consistency (WPEC)*, 2019.

[21] S. Sadhu, “Conflict-Free Knowledge Graph Projection: A Three-Phase CRDT Pipeline for Multi-Agent Memory Systems,” preprint, 2026.
